

Міністерство освіти і науки України
Вінницький національний технічний університет
Факультет інформаційних технологій та комп'ютерної інженерії
Кафедра ПЗ

Практична робота №2
з дисципліни «Розробка проєктів засобами платформи .NET»

Виконав: ст. 1ПІ-24Б

Перевірів: викл.

Кузя Н.І.

Позур М.Ю.

Тема: ООП та обробка виключень у консольному застосунку.

Мета: Отримати практичні навички роботи з класами в C# з використанням принципів ООП (інкапсуляція, успадкування, поліморфізм, абстракція), роботи з інтерфейсами та коректної обробки виключень (Exception).

Завдання (Варіант 3. Співробітники компанії)

Створити консольний проєкт (.NET). Спроектувати ієрархію класів: базовий клас Employee та похідні Manager, Developer. Реалізувати інкапсуляцію з валідацією властивостей (заборона від'ємної зарплати), інтерфейс IComparable<Employee>, власний клас винятку. Створити поліморфну колекцію та просте меню для управління співробітниками (додавання, перегляд, розрахунок сумарного фонду заробітної плати, сортування).

Хід роботи

Реалізовано ієрархію класів зі спільним базовим абстрактним класом Employee. Метод CalculateSalary() є віртуальним і перевизначається у класах Manager та Developer. Інтерфейс IComparable реалізовано для сортування співробітників за їхньою підсумковою зарплатою у порядку спадання. Властивість BaseSalary містить перевірку: якщо значення менше або дорівнює нулю, генерується власний виняток InvalidSalaryException, який безпечно обробляється у блоці try-catch-finally в основному меню.

Лістинг програми (Program.cs)

```
using System;
using System.Collections.Generic;

namespace CompanyEmployeesApp
{
    public class InvalidSalaryException : Exception
    {
        public InvalidSalaryException() { }
        public InvalidSalaryException(string message) : base(message) { }
        public InvalidSalaryException(string message, Exception inner) : base(message,
inner) { }
    }

    public enum DeveloperLevel
    {
        Junior = 1,
        Middle = 2,
        Senior = 3
    }

    public abstract class Employee : IComparable<Employee>
```

```

{
    private string _fullName = string.Empty;
    private double _baseSalary;

    public string FullName
    {
        get => _fullName;
        set => _fullName = string.IsNullOrWhiteSpace(value)
            ? throw new ArgumentException("Ім'я не може бути порожнім.")
            : value;
    }

    public double BaseSalary
    {
        get => _baseSalary;
        set
        {
            if (value <= 0)
            {
                throw new InvalidSalaryException("Базова зарплата повинна бути більшою
за нуль.");
            }
            _baseSalary = value;
        }
    }

    protected Employee(string fullName, double baseSalary)
    {
        FullName = fullName;
        BaseSalary = baseSalary;
    }

    public virtual double CalculateSalary() => BaseSalary;

    public virtual void PrintInfo()
    {
        Console.WriteLine($"{FullName} | Базова ставка: {BaseSalary:C2} | До виплати:
{CalculateSalary():C2}");
    }

    public int CompareTo(Employee? other)
    {
        if (other == null) return 1;
        return other.CalculateSalary().CompareTo(this.CalculateSalary());
    }
}

public class Manager : Employee
{
    private int _subordinatesCount;
    private const double BonusPerSubordinate = 1500.0;

    public int SubordinatesCount
    {
        get => _subordinatesCount;
        set => _subordinatesCount = value >= 0 ? value : throw new
ArgumentException("Кількість підлеглих не може бути від'ємною.");
    }

    public Manager(string fullName, double baseSalary, int subordinatesCount)
        : base(fullName, baseSalary)
    {
        SubordinatesCount = subordinatesCount;
    }

    public override double CalculateSalary()
    {
        return base.CalculateSalary() + (SubordinatesCount * BonusPerSubordinate);
    }

    public override void PrintInfo()
    {
        Console.Write("[Менеджер] ");
        base.PrintInfo();
    }
}

```

```

        Console.WriteLine($"      L Підлеглих: {SubordinatesCount} (Бонус:
{SubordinatesCount * BonusPerSubordinate:C2})");
    }
}

public class Developer : Employee
{
    public DeveloperLevel Level { get; set; }

    public Developer(string fullName, double baseSalary, DeveloperLevel level)
        : base(fullName, baseSalary)
    {
        Level = level;
    }

    public override double CalculateSalary()
    {
        double levelBonus = Level switch
        {
            DeveloperLevel.Junior => 0,
            DeveloperLevel.Middle => 5000,
            DeveloperLevel.Senior => 10000,
            _ => 0
        };
        return base.CalculateSalary() + levelBonus;
    }

    public override void PrintInfo()
    {
        Console.Write($"[Розробник {Level}] ");
        base.PrintInfo();
    }
}

internal class Program
{
    static void Main()
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;

        List<Employee> employees = new List<Employee>
        {
            new Manager("Іван Коваленко", 20000, 5),
            new Developer("Олена Петрівна", 18000, DeveloperLevel.Middle),
            new Developer("Олександр Сидорчук", 25000, DeveloperLevel.Senior)
        };

        bool isRunning = true;
        while (isRunning)
        {
            Console.WriteLine("\n=== Співробітники компанії ===");
            Console.WriteLine("1. Додати менеджера");
            Console.WriteLine("2. Додати розробника");
            Console.WriteLine("3. Показати список співробітників");
            Console.WriteLine("4. Сумарний фонд заробітної плати");
            Console.WriteLine("5. Відсортувати за зарплатою (спадання)");
            Console.WriteLine("0. Вихід");

            int choice = ReadInt("Ваш вибір: ", 0, 5);

            isRunning = choice switch
            {
                1 => HandleAddManager(employees),
                2 => HandleAddDeveloper(employees),
                3 => HandleShowAll(employees),
                4 => HandleShowTotalFund(employees),
                5 => HandleSortEmployees(employees),
                0 => false,
                _ => true
            };
        }
        Console.WriteLine("\nРоботу завершено.");
    }
}

```

```

static bool HandleAddManager(List<Employee> employees)
{
    Console.WriteLine("\n--- Додавання менеджера ---");
    try
    {
        string name = ReadString("Введіть ПІБ: ");
        double salary = ReadDouble("Базова зарплата: ");
        int subs = ReadInt("Кількість підлеглих: ", 0, 1000);

        Manager newManager = new Manager(name, salary, subs);
        employees.Add(newManager);
        Console.WriteLine("Менеджера успішно додано.");
    }
    catch (InvalidSalaryException ex)
    {
        Console.ForegroundColor = ConsoleColor.Red;
        Console.WriteLine($"Помилка валідації зарплати: {ex.Message}");
        Console.ResetColor();
    }
    catch (Exception ex)
    {
        Console.ForegroundColor = ConsoleColor.Red;
        Console.WriteLine($"Непередбачена помилка: {ex.Message}");
        Console.ResetColor();
    }
    finally
    {
        Console.WriteLine("Операцію додавання завершено (finally блок).");
    }
    return true;
}

static bool HandleAddDeveloper(List<Employee> employees)
{
    Console.WriteLine("\n--- Додавання розробника ---");
    try
    {
        string name = ReadString("Введіть ПІБ: ");
        double salary = ReadDouble("Базова зарплата: ");

        Console.WriteLine("Оберіть рівень (1 - Junior, 2 - Middle, 3 - Senior):");
        int levelInt = ReadInt("Рівень: ", 1, 3);
        DeveloperLevel level = (DeveloperLevel)levelInt;

        Developer newDev = new Developer(name, salary, level);
        employees.Add(newDev);
        Console.WriteLine("Розробника успішно додано.");
    }
    catch (InvalidSalaryException ex)
    {
        Console.ForegroundColor = ConsoleColor.Red;
        Console.WriteLine($"Помилка валідації зарплати: {ex.Message}");
        Console.ResetColor();
    }
    catch (Exception ex)
    {
        Console.ForegroundColor = ConsoleColor.Red;
        Console.WriteLine($"Непередбачена помилка: {ex.Message}");
        Console.ResetColor();
    }
    finally
    {
        Console.WriteLine("Операцію додавання завершено (finally блок).");
    }
    return true;
}

static bool HandleShowAll(List<Employee> employees)
{
    Console.WriteLine("\n--- Список співробітників ---");
    if (employees.Count == 0)
    {
        Console.WriteLine("Список порожній.");
        return true;
    }
}

```

```

    }

    foreach (Employee emp in employees)
    {
        emp.PrintInfo();
    }
    return true;
}

static bool HandleShowTotalFund(List<Employee> employees)
{
    double total = 0;
    foreach (var emp in employees)
    {
        total += emp.CalculateSalary();
    }
    Console.WriteLine($"Сумарний фонд заробітної плати: {total:C2}");
    return true;
}

static bool HandleSortEmployees(List<Employee> employees)
{
    employees.Sort();
    Console.WriteLine("\nСпівробітників відсортовано за зарплатою (від
найбільшої).");
    HandleShowAll(employees);
    return true;
}

static string ReadString(string prompt)
{
    while (true)
    {
        Console.Write(prompt);
        string? input = Console.ReadLine();
        if (!string.IsNullOrEmpty(input)) return input;
        Console.WriteLine("Рядок не може бути порожнім.");
    }
}

static double ReadDouble(string prompt)
{
    while (true)
    {
        Console.Write(prompt);
        if (double.TryParse(Console.ReadLine()?.Replace(',', '.'),
System.Globalization.NumberStyles.Any, System.Globalization.CultureInfo.InvariantCulture,
out double value))
            return value;
        Console.WriteLine("Введіть коректне число.");
    }
}

static int ReadInt(string prompt, int min, int max)
{
    while (true)
    {
        Console.Write(prompt);
        if (int.TryParse(Console.ReadLine(), out int value) && value >= min && value
<= max)
            return value;
        Console.WriteLine($"Введіть ціле число від {min} до {max}.");
    }
}
}
}

```

Посилання на GitHub

Репозиторій із вихідним кодом: <https://github.com/shwballl/vntu-dotnet>

```

=== Співробітники компанії ===
1. Додати менеджера
2. Додати розробника
3. Показати список співробітників
4. Сумарний фонд заробітної плати
5. Відсортувати за зарплатою (спадання)
0. Вихід
Ваш вибір: 1

--- Додавання менеджера ---
Введіть ПІБ: Барчук Олег Іванович
Базова зарплата: 400
Кількість підлеглих: 2
Менеджера успішно додано.
Операцію додавання завершено (finally блок).

=== Співробітники компанії ===
1. Додати менеджера
2. Додати розробника
3. Показати список співробітників
4. Сумарний фонд заробітної плати
5. Відсортувати за зарплатою (спадання)
0. Вихід
Ваш вибір: 2

--- Додавання розробника ---
Введіть ПІБ: Антонов Василь Васильович
Базова зарплата: 600
Оберіть рівень (1 - Junior, 2 - Middle, 3 - Senior):
Рівень: 1
Розробника успішно додано.
Операцію додавання завершено (finally блок).

=== Співробітники компанії ===
1. Додати менеджера
2. Додати розробника
3. Показати список співробітників
4. Сумарний фонд заробітної плати
5. Відсортувати за зарплатою (спадання)
0. Вихід
Ваш вибір: 3

--- Список співробітників ---
[Менеджер] Іван Коваленко | Базова ставка: 20 000,00 ₴ | До виплати: 27 500,00 ₴
└ Підлеглих: 5 (Бонус: 7 500,00 ₴)
[Розробник Middle] Олена Петрівна | Базова ставка: 18 000,00 ₴ | До виплати: 23 000,00 ₴
[Розробник Senior] Олександр Сидорчук | Базова ставка: 25 000,00 ₴ | До виплати: 35 000,00 ₴
[Менеджер] Барчук Олег Іванович | Базова ставка: 400,00 ₴ | До виплати: 3 400,00 ₴
└ Підлеглих: 2 (Бонус: 3 000,00 ₴)
[Розробник Junior] Антонов Василь Васильович | Базова ставка: 600,00 ₴ | До виплати: 600,00 ₴

=== Співробітники компанії ===
1. Додати менеджера
2. Додати розробника
3. Показати список співробітників
4. Сумарний фонд заробітної плати
5. Відсортувати за зарплатою (спадання)
0. Вихід
Ваш вибір: 4

Сумарний фонд заробітної плати: 89 500,00 ₴

=== Співробітники компанії ===
1. Додати менеджера
2. Додати розробника
3. Показати список співробітників
4. Сумарний фонд заробітної плати
5. Відсортувати за зарплатою (спадання)
0. Вихід
Ваш вибір: 5

Співробітників відсортовано за зарплатою (від найбільшої).

--- Список співробітників ---
[Розробник Senior] Олександр Сидорчук | Базова ставка: 25 000,00 ₴ | До виплати: 35 000,00 ₴
[Менеджер] Іван Коваленко | Базова ставка: 20 000,00 ₴ | До виплати: 27 500,00 ₴
└ Підлеглих: 5 (Бонус: 7 500,00 ₴)
[Розробник Middle] Олена Петрівна | Базова ставка: 18 000,00 ₴ | До виплати: 23 000,00 ₴
[Менеджер] Барчук Олег Іванович | Базова ставка: 400,00 ₴ | До виплати: 3 400,00 ₴
└ Підлеглих: 2 (Бонус: 3 000,00 ₴)
[Розробник Junior] Антонов Василь Васильович | Базова ставка: 600,00 ₴ | До виплати: 600,00 ₴

```

Рисунок 1 – Результат роботи програми

Відповіді на контрольні питання

1. У чому різниця між абстрактним класом та інтерфейсом? Коли що застосовувати?

Абстрактний клас може містити реалізацію методів і поля, але не підтримує множинне успадкування. Інтерфейс містить лише сигнатури (до C# 8) і підтримує множинне успадкування. Абстрактні класи застосовують для спільної базової логіки споріднених об'єктів, а інтерфейси – для задання спільного контракту поведінки різнотипних об'єктів.

2. Навіщо потрібна інкапсуляція і чому властивість з валідацією в `set` краще за публічне поле?

Інкапсуляція приховує внутрішній стан об'єкта та захищає його від некоректних змін ззовні. Властивість із валідацією в `set` дозволяє перевірити дані перед присвоєнням (наприклад, заборонити від'ємну зарплату) і згенерувати виняток, чого публічне поле зробити не може.

3. Навіщо потрібне перевантаження конструкторів, якщо можна обійтися одним конструктором з параметрами за замовчуванням?

Перевантаження конструкторів дозволяє створювати об'єкти різними способами (з різним набором вхідних даних), адаптуючись під різні сценарії ініціалізації. Це гнучкіше, ніж параметри за замовчуванням, оскільки дозволяє виконувати різну логіку або викликати різні базові конструктори в залежності від сигнатури.

4. Як реалізується поліморфізм у C# на рівні виконання (пізніше зв'язування)? Наведіть приклад із власної ієрархії.

Він реалізується за допомогою віртуальних (`virtual/abstract`) методів та їх перевизначення (`override`) у похідних класах. Під час виконання програма визначає фактичний тип об'єкта в пам'яті і викликає відповідний метод. Наприклад, виклик `emp.CalculateSalary()` для елемента колекції типу `Employee` викличе або логіку `Manager`, або `Developer`.

5. У чому різниця між `abstract`-методом і `virtual`-методом з реалізацією за замовчуванням?

`abstract`-метод не має реалізації в базовому класі і обов'язково має бути перевизначений у всіх похідних класах (його клас також має бути `abstract`).

virtual-метод має базову реалізацію, яку похідний клас може або використати, або добровільно перевизначити.

6. Для чого призначений інтерфейс `Comparable<T>` і як його реалізація впливає на роботу `List<T>.Sort()`?

`Comparable<T>` дозволяє задати «природне» правило порівняння двох об'єктів певного типу через метод `CompareTo`. Коли клас його реалізує, метод `List<T>.Sort()` знає, як впорядкувати елементи колекції без передачі додаткових компараторів.

7. Що виконується у блоці `finally` і в яких випадках це критично важливо?

Код у блоці `finally` виконується завжди, незалежно від того, чи стався виняток у блоці `try`, чи ні. Це критично важливо для звільнення зовнішніх ресурсів (закриття файлів, з'єднань із БД), щоб уникнути витоків пам'яті.

8. У чому різниця між обробкою винятку в `catch (SpecificException)` і `catch (Exception)`? Чому порядок гілок `catch` має значення?

`catch (SpecificException)` перехоплює лише конкретний тип помилки, дозволяючи специфічно на неї відреагувати. `catch (Exception)` перехоплює абсолютно всі помилки. Порядок важливий: спочатку пишуть конкретні винятки, а `Exception` залишають у кінці як запасний варіант (інакше конкретні `catch` ніколи не виконаються).

9. Навіщо створювати власні класи винятків, якщо є вбудовані?

Власні класи винятків (наприклад, `InvalidSalaryException`) додають доменну семантику, роблячи код більш зрозумілим, і дозволяють точніше перехоплювати специфічні бізнес-помилки у блоках `catch`, відділяючи їх від системних помилок.

10. Чому власний клас винятку прийнято постачати трьома стандартними конструкторами?

Це офіційна рекомендація Microsoft. Порожній конструктор – для базового викидання; з повідомленням – для опису помилки; з повідомленням та внутрішнім винятком (`inner exception`) – для збереження ланцюжка помилок (стеку викликів), якщо новий виняток генерується на основі іншого.

11. Що станеться, якщо колекція базового типу міститиме похідний об'єкт, а виклик методу не позначено `virtual/override`?

Станеться «приховування» (shadowing). Викличеться реалізація методу базового класу, оскільки компілятор орієнтуватиметься на тип посилання (Employee), а не на фактичний тип об'єкта (Manager), тому поліморфної поведінки не буде.

12. Чому колекцію об'єктів ієрархії варто типізувати базовим класом (List<Shape>), а не окремими списками для кожного похідного типу?

Це реалізація принципу абстракції і поліморфізму. Єдина колекція базового типу (List<Employee>) дозволяє застосовувати єдиний цикл foreach для обробки всіх похідних об'єктів одночасно, значно зменшуючи дублювання коду та спрощуючи архітектуру.

Висновок: під час виконання практичної роботи №2 було створено консольний застосунок на платформі .NET з використанням принципів ООП. Спроектовано ієрархію співробітників (абстрактний базовий клас та класи-спадкоємці з віртуальними методами), реалізовано поліморфну колекцію та сортування через IComparable. Також було розроблено власний клас винятку для контролю цілісності даних (заборона нульової/від'ємної зарплати), обробка якого безпечно виконується в структурі try-catch-finally.